Abstract Communications

RTL Handshaking

Zainalabedin Navabi

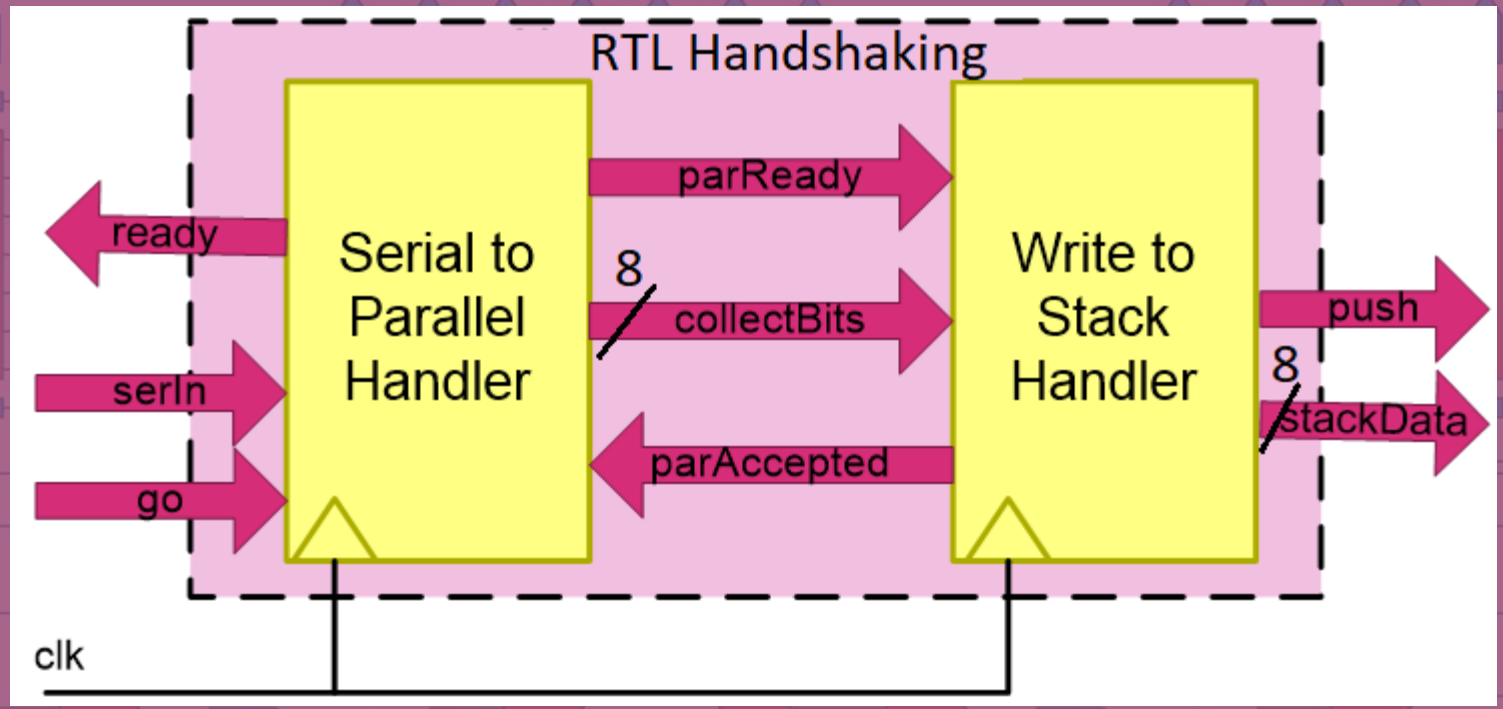
Slides prepared by Nooshin Nosrati

Outline

- Handshaking
  - RTL Handshaking
  - Abstract Handshaking

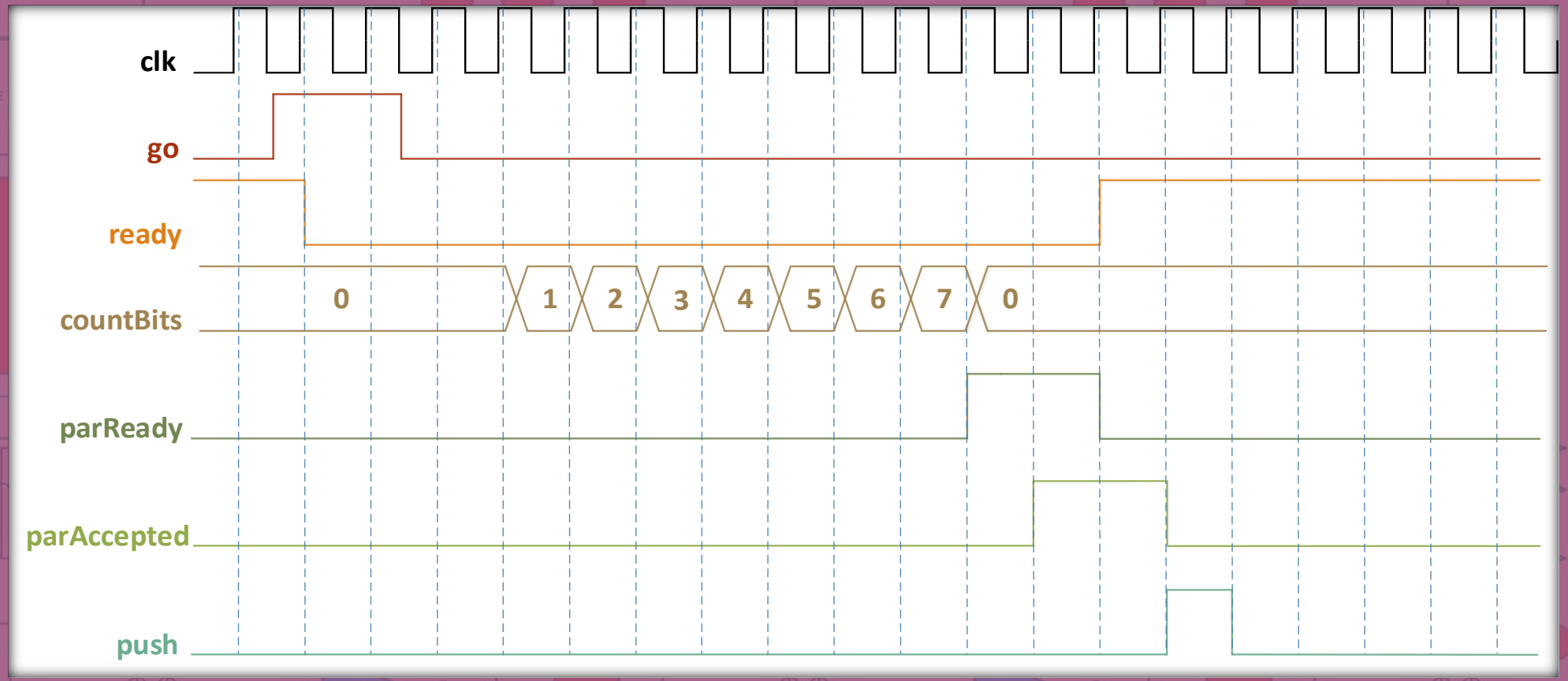
RTL Handshaking

- Serial to parallel - block diagram



RTL Handshaking

- Serial to parallel - timing



RTL Handshaking

RTL Handshaking_Main.cpp | RTL Handshaking_TB.cpp | RTL Handshaking_TB.h

RTL Handshaking | (Global Scope)

```
1   #include <systemc.h>
2
3   SC_MODULE(RTLhandshaking) {
4
5       sc_in<sc_logic> clk, go;
6       sc_out<sc_logic> ready;
7       sc_in<sc_logic> serIn;
8       sc_out<sc_lv<8>> stackData;
9       sc_out<sc_logic> push;
10
11      sc_signal<sc_lv<8>> collectBits;
12      sc_signal<sc_logic> parAccepted, parReady;
13
14      // Serial-to-Parallel internals
15      enum S2PstateType {go1Wait, go0Wait, count8Bits, got8Bits};
16      sc_signal<S2PstateType> S2Pstate;
17      sc_signal<sc_lv<3>> countBits;
18
19      // Write-to-Stack internals
20      enum W2SstateType {par1Wait, par0Wait, pushData};
21      sc_signal<W2SstateType> W2Sstate;
```

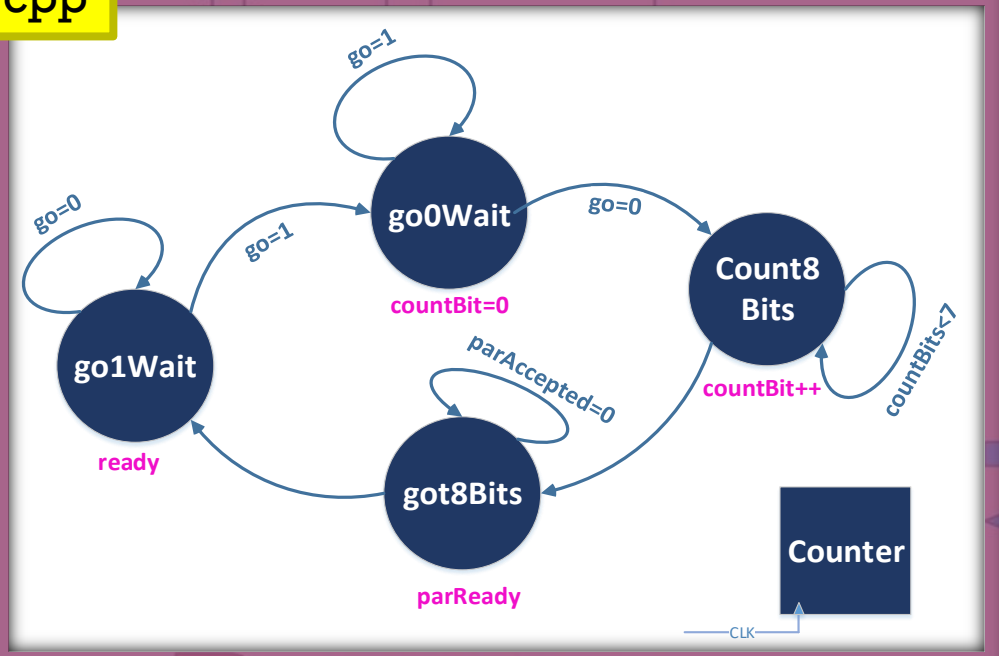
RTL Handshaking.h

```
23      SC_CTOR(RTLhandshaking) {
24          SC_METHOD(S2Ptransition);
25              sensitive << clk;
26          SC_METHOD(S2Psignals);
27              sensitive << S2Pstate;
28          SC_METHOD(W2Stransition);
29              sensitive << clk;
30          SC_METHOD(W2Ssignals);
31              sensitive << W2Sstate;
32      }
33      void S2Ptransition();
34      void S2Psignals();
35      void W2Stransition();
36      void W2Ssignals();
37  };
```

RTL Handshaking

RTL Handshaking.cpp

```
3  void RTLhandshaking::S2Ptransition() {
4      if (clk->event() && clk == '1')
5          switch (S2Pstate) {
6          case go1Wait:
7              if (go == '1') S2Pstate = go0Wait;
8              else S2Pstate = go1Wait;
9              break;
10         case go0Wait:
11             if (go == '0') S2Pstate = count8Bits;
12             else S2Pstate = go0Wait;
13             countBits = 0;
14             break;
15         case count8Bits:
16             if (countBits.read().to_uint() == 7) S2Pstate = got8Bits;
17             else S2Pstate = count8Bits;
18             collectBits = (serIn, collectBits.read().range(7, 1));
19             countBits = countBits.read().to_uint() + 1;
20             break;
21         case got8Bits:
22             if (parAccepted == '1') S2Pstate = go1Wait;
23             else S2Pstate = got8Bits;
24             break;
25         default:
26             break;
27     }
28 }
```

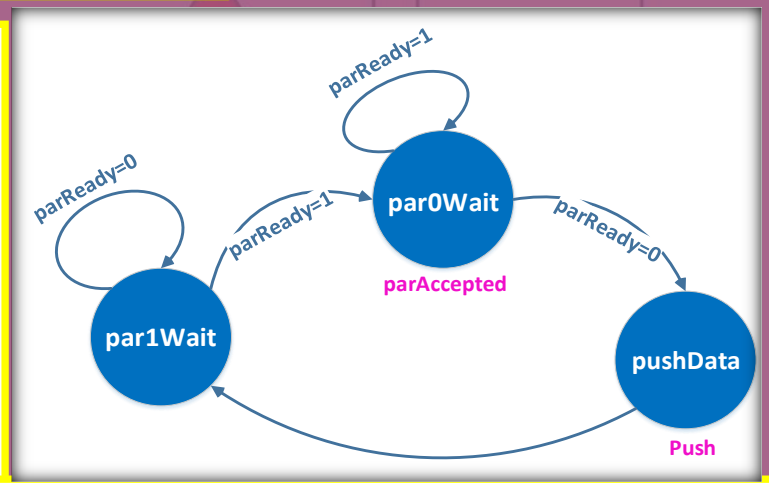



```
29 void RTLhandshaking::S2Psignals() {
30     ready = SC_LOGIC_0;
31     parReady = SC_LOGIC_0;
32     if (S2Pstate == go1Wait) ready = SC_LOGIC_1;
33     else if (S2Pstate == got8Bits) parReady = SC_LOGIC_1;
34 }
35
```

RTL Handshaking

RTL Handshaking.cpp

```
36  void RTLhandshaking::W2Stransition() {
37      if (clk->event() && clk == '1')
38          switch (W2Sstate) {
39          case par1Wait:
40              if (parReady == '1') W2Sstate = par0Wait;
41              else W2Sstate = par1Wait;
42              break;
43          case par0Wait:
44              if (parReady == '0') W2Sstate = pushData;
45              else W2Sstate = par0Wait;
46              break;
47          case pushData:
48              W2Sstate = par1Wait;
49              break;
50          default:
51              break;
52      }
53  }
```




```
54  void RTLhandshaking::W2Ssignals() {
55      push = SC_LOGIC_0;
56      parAccepted = SC_LOGIC_0;
57      stackData = 0;
58      if (W2Sstate == par0Wait) {
59          parAccepted = SC_LOGIC_1;
60          stackData = collectBits;
61      }
62      else if (W2Sstate == pushData) {
63          push = SC_LOGIC_1;
64          stackData = collectBits;
65      }
66  }
```

RTL Handshaking

RTL Handshaking.cpp | RTL Handshaking_TB.h | RTL Handshaking_TB.cpp

RTL Handshaking | (Global Scope)

RTL Handshaking_TB.h

```
1   #include "RTL Handshaking.h"
2
3   SC_MODULE (RTLhandshakingTB) {
4       sc_signal<sc_logic> clk;
5       sc_signal<sc_logic> go, serIn;
6       sc_signal<sc_logic> ready;
7       sc_signal<sc_lv<8>> parOut;
8       sc_signal<sc_logic> push;
9
10      RTLhandshaking* S2W; // Ser to par and Write to stack
11
12      SC_CTOR(RTLhandshakingTB) {
13          S2W = new RTLhandshaking("Handshaking_TB");
14          S2W -> clk(clk);
15          S2W -> go(go);
16          S2W -> serIn(serIn);
17          S2W -> ready(ready);
18          S2W -> stackData(parOut);
19          S2W -> push(push);
20
21          SC_THREAD(clocking);
22          SC_THREAD(serialData);
23      }
24      void clocking();
25      void serialData();
26  };
```

RTL Handshaking

RTL Handshaking_TB.cpp

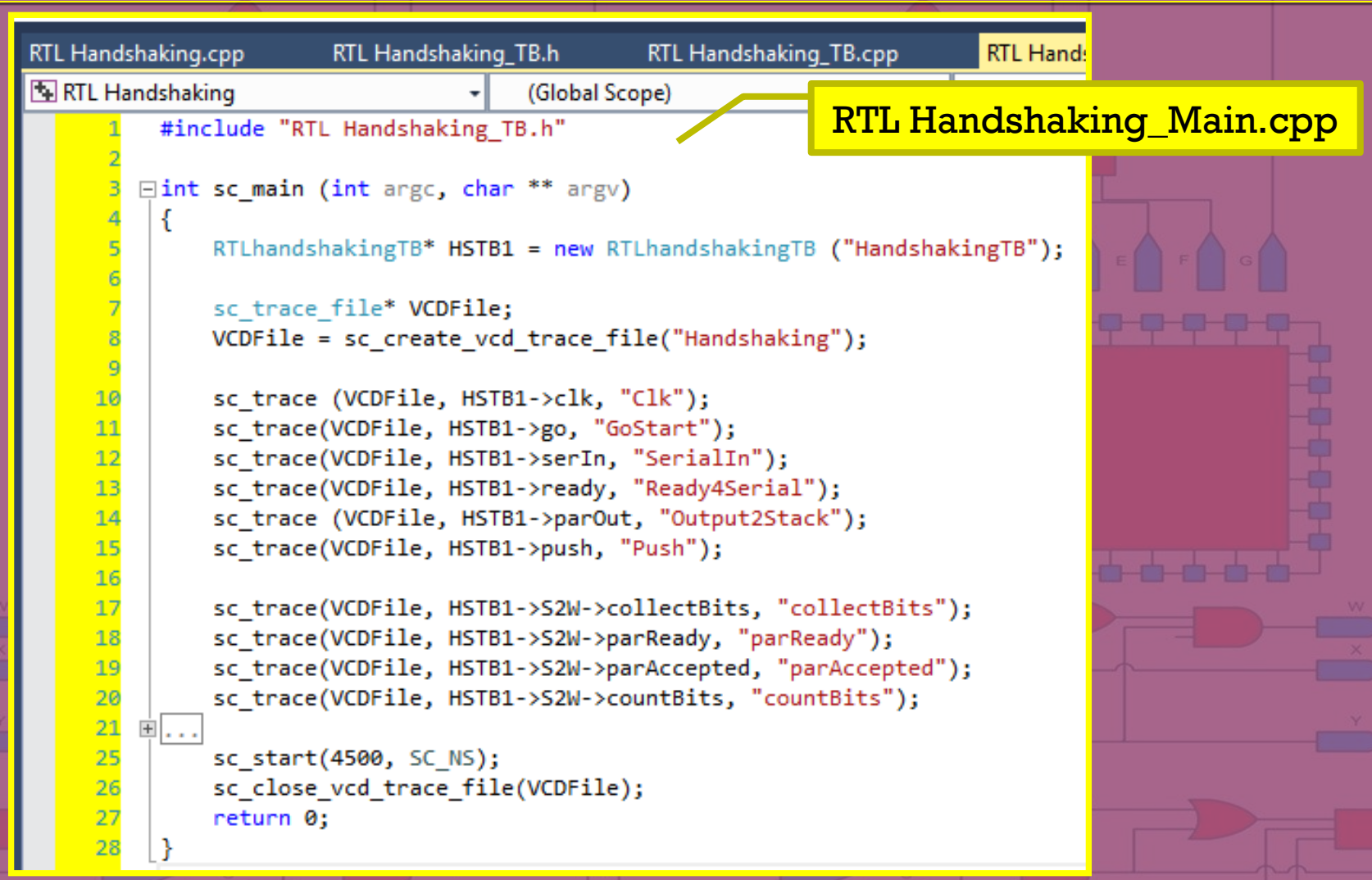
RTL Handshaking.h

RTL Handshaking (Global Sco

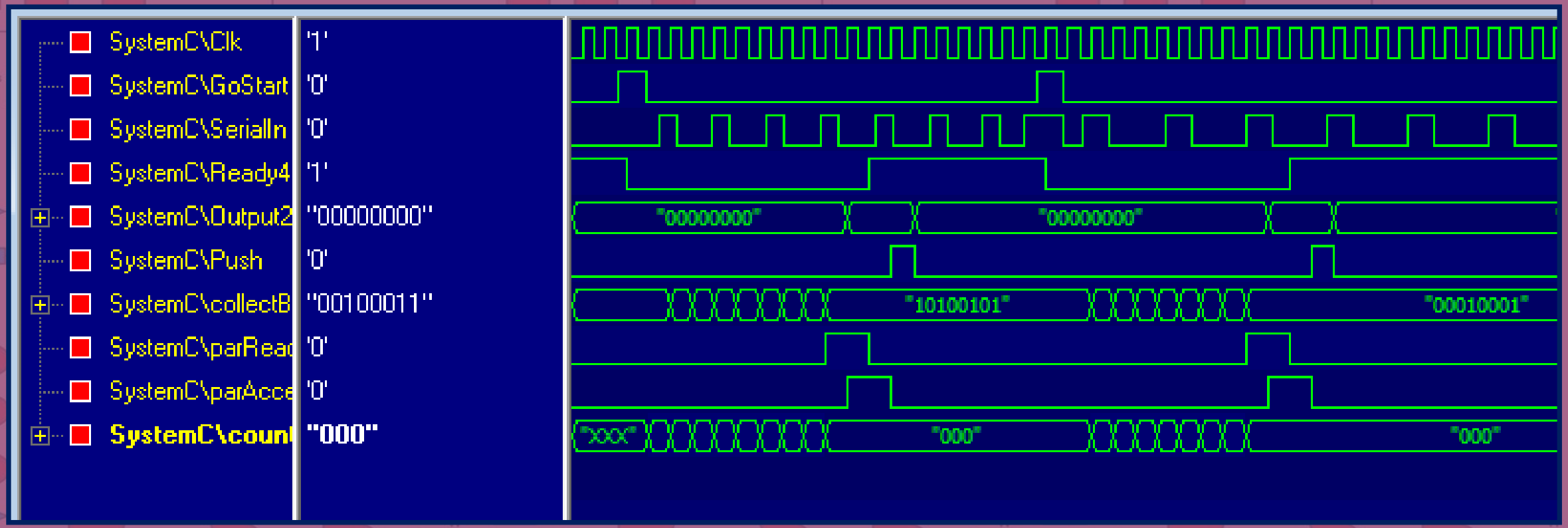
```
1   #include "RTL Handshaking_TB.h"
2
3   void RTLhandshakingTB::clocking() {
4       int i;
5       clk = SC_LOGIC_0;
6       for (i = 0; i<=500; i++){
7           wait (29, SC_NS);
8           clk = SC_LOGIC_1;
9           wait (29, SC_NS);
10          clk = SC_LOGIC_0;
11      }
12  }
```

```
14  void RTLhandshakingTB::serialData() {
15      int i, j;
16      go = SC_LOGIC_0;
17      serIn = SC_LOGIC_0;
18      for (i = 1; i < 5; i++){
19          if (ready == SC_LOGIC_1){
20              wait(31, SC_NS);
21              go = SC_LOGIC_1;
22              wait(73, SC_NS);
23              go = SC_LOGIC_0;
24              for (j = 0; j < 7; j++){
25                  serIn = SC_LOGIC_0;
26                  wait(17 * i, SC_NS);
27                  serIn = SC_LOGIC_1;
28                  wait(23 * i, SC_NS);
29                  serIn = SC_LOGIC_0;
30                  wait(31 * i, SC_NS);
31                  serIn = SC_LOGIC_1;
32              }
33          }
34          else wait(91, SC_NS);
35      }
36  }
```

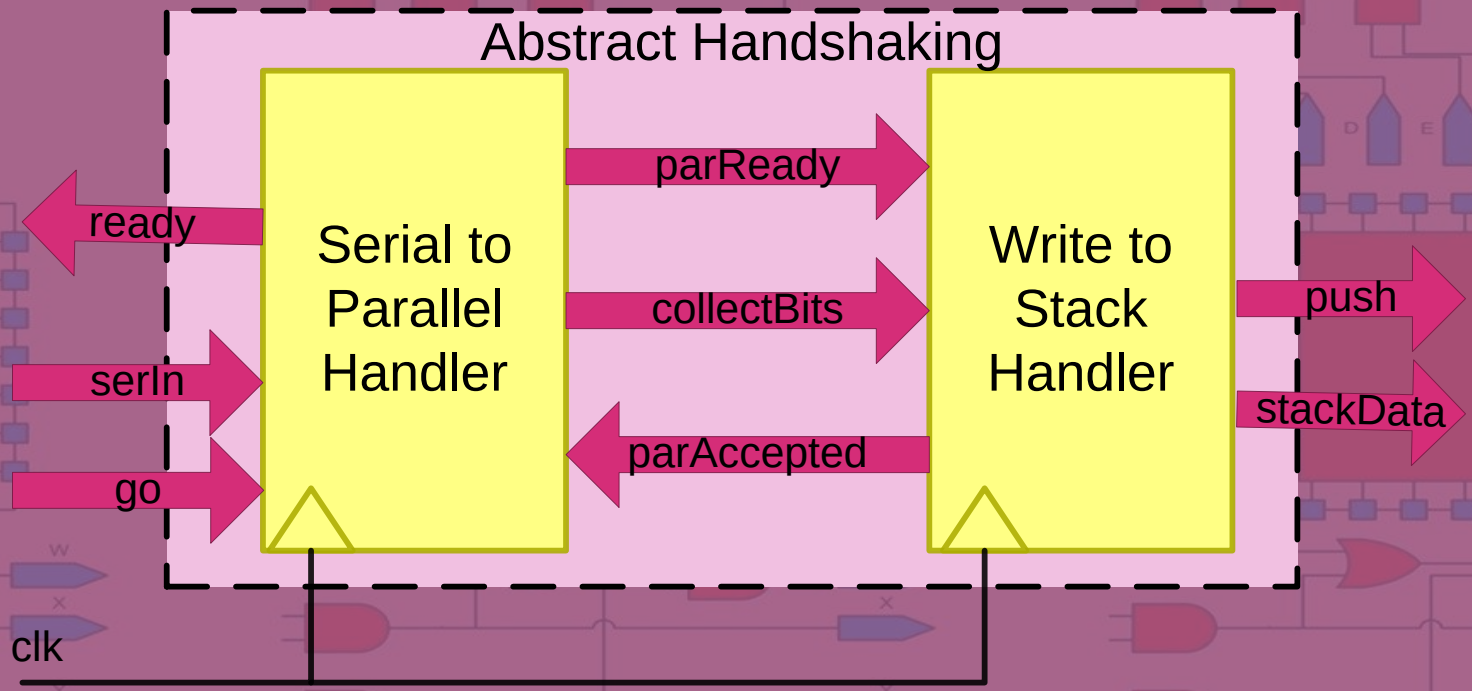
RTL Handshaking



RTL Handshaking



Abstract Handshaking



Abstract Handshaking

Abstract Handshaking .h

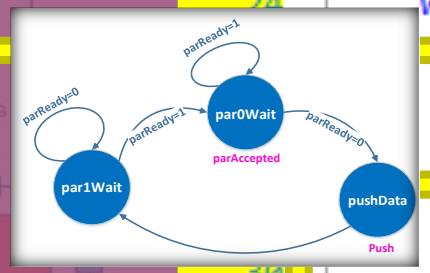
```
1   #include <systemc.h>
2
3   SC_MODULE(AbstractHandshaking) {
4
5       sc_in<sc_logic> clk, go;
6       sc_out<sc_logic> ready;
7       sc_in<sc_logic> serIn;
8       sc_out<sc_lv<8>> stackData;
9       sc_out<sc_logic> push;
10
11      sc_lv<8> collectBits;
12      sc_event parAccepted_ev, parReady_ev;
13
14      SC_CTOR(AbstractHandshaking) {
15          SC_THREAD(S2Phandler);
16              sensitive << clk.pos();
17          SC_THREAD(W2Shandler);
18              sensitive << clk.pos();
19      }
20      void S2Phandler();
21      void W2Shandler();
22  };
```

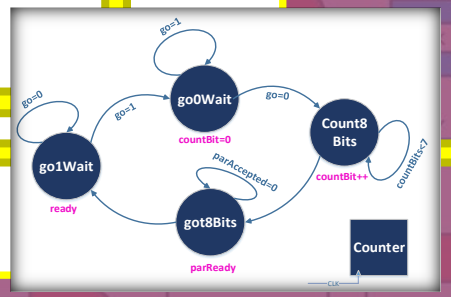
Abstract Handshaking

handshaking.cpp

```cpp
#include "Abstract Handshaking.h"

void AbstractHandshaking::S2Phandler() {
    int i;
    ready = SC_LOGIC_0;
    while (1){
        ready = SC_LOGIC_1;
        while (go != '1') wait();
        ready = SC_LOGIC_0;
        while (go != '0') wait();
        for (i = 0; i < 8; i++){
            wait();
            collectBits[i] = serIn;
        }
        wait();
        parReady_ev.notify();
        wait(parAccepted_ev);
    }
}
```

```cpp
void AbstractHandshaking::W2Shandler() {
    push = SC_LOGIC_0;
    stackData = 0;
    while (1){
        wait(parReady_ev);
        stackData = collectBits;
        wait();
        parAccepted_ev.notify();
        wait();
        push = SC_LOGIC_1;
        wait();
        push = SC_LOGIC_0;
        stackData = 0;
    }
}
```





Abstract Handshaking

Abstract Handshaking_TB.h

```
#include "Abstract Handshaking.h"

SC_MODULE(AbstractHandshakingTB) {
    sc_signal<sc_logic> clk;
    sc_signal<sc_logic> go, serIn;
    sc_signal<sc_logic> ready;
    sc_signal<sc_lv<8>> parOut;
    sc_signal<sc_logic> push;

    AbstractHandshaking* S2W; // Ser to par and Write to stack

    SC_CTOR(AbstractHandshakingTB) {
        S2W = new AbstractHandshaking("Handshaking_TB");
        S2W -> clk(clk);
        S2W -> go(go);
        S2W -> serIn(serIn);
        S2W -> ready(ready);
        S2W -> stackData(parOut);
        S2W -> push(push);

        SC_THREAD(clocking);
        SC_THREAD(serialData);
    }
    void clocking();
    void serialData();
};
```

Abstract Handshaking_TB.cpp

```
#include "Abstract Handshaking_TB.h"

void AbstractHandshakingTB::clocking() {
    int i;
    clk = SC_LOGIC_0;
    for (i = 0; i<=500; i++){
        wait (29, SC_NS);
        clk = SC_LOGIC_1;
        wait (29, SC_NS);
        clk = SC_LOGIC_0;
    }
}
```

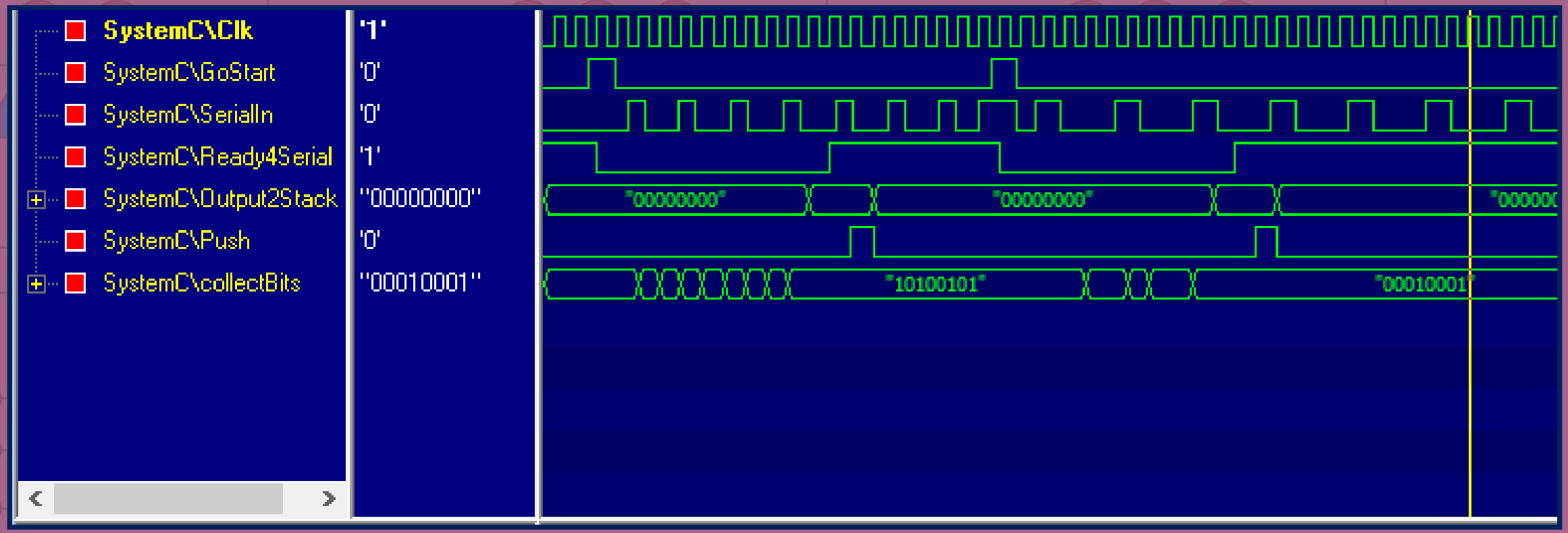
```
void AbstractHandshakingTB::serialData() {
    int i, j;
    go = SC_LOGIC_0;
    serIn = SC_LOGIC_0;
    for (i = 1; i < 5; i++){
        if (ready == SC_LOGIC_1){
            wait(31, SC_NS);
            go = SC_LOGIC_1;
            wait(73, SC_NS);
            go = SC_LOGIC_0;
            for (j = 0; j < 7; j++){
                serIn = SC_LOGIC_0;
                wait(17 * i, SC_NS);
                serIn = SC_LOGIC_1;
                wait(23 * i, SC_NS);
                serIn = SC_LOGIC_0;
                wait(31 * i, SC_NS);
                serIn = SC_LOGIC_1;
            }
        }
        else wait(91, SC_NS);
    }
}
```

Abstract Handshaking

Abstract Handshaking_Main.cpp

```
#include "Abstract Handshaking_TB.h"

int sc_main (int argc, char ** argv)
{
    AbstractHandshakingTB* HSTB1 = new AbstractHandshakingTB("HandshakingTB");

    sc_trace_file* VCDFile;
    VCDFile = sc_create_vcd_trace_file("Handshaking");

    sc_trace (VCDFile, HSTB1->clk, "Clk");
    sc_trace(VCDFile, HSTB1->go, "GoStart");
    sc_trace(VCDFile, HSTB1->serIn, "SerialIn");
    sc_trace(VCDFile, HSTB1->ready, "Ready4Serial");
    sc_trace (VCDFile, HSTB1->parOut, "Output2Stack");
    sc_trace(VCDFile, HSTB1->push, "Push");

    sc_trace(VCDFile, HSTB1->S2W->collectBits, "collectBits");

    sc_start(4500, SC_NS);
    sc_close_vcd_trace_file(VCDFile);
    return 0;
}
```

Abstract Handshaking



Conclusions

- We have covered these topics:
  - RTL handshaking
  - Serial to parallel communication
  - Abstract handshaking
  - Using wait()-notify()

Acknowledgment

- Slides are contributed by Mahsa Akhsham